

CareerKartos

A job discovery platform that indexes listings directly from employer career pages.

Live: careerkartos.vercel.app · **API:** careerkartos-api.onrender.com · **Repository:** github.com/rjayswal-pythonista/careerkartos
Author: Roshan Jayswal · **Report date:** 13 September 2026

1. What the product does

CareerKartos collects job listings once a day from the applicant tracking systems that employers already publish, normalises them into a single searchable schema, and presents them as one feed. Every listing links back to the employer's own posting.

The product boundary, which the rest of the design follows from: this is a discovery and indexing layer. It does not accept applications, host application forms, or submit anything on a user's behalf. That boundary keeps the product clear of employer Terms of Service restrictions, and it is enforced structurally — there is no submit endpoint that could accidentally grow one. A test asserts its absence.

3,129
LINES OF PYTHON

17
API ENDPOINTS

6
ATS CONNECTORS

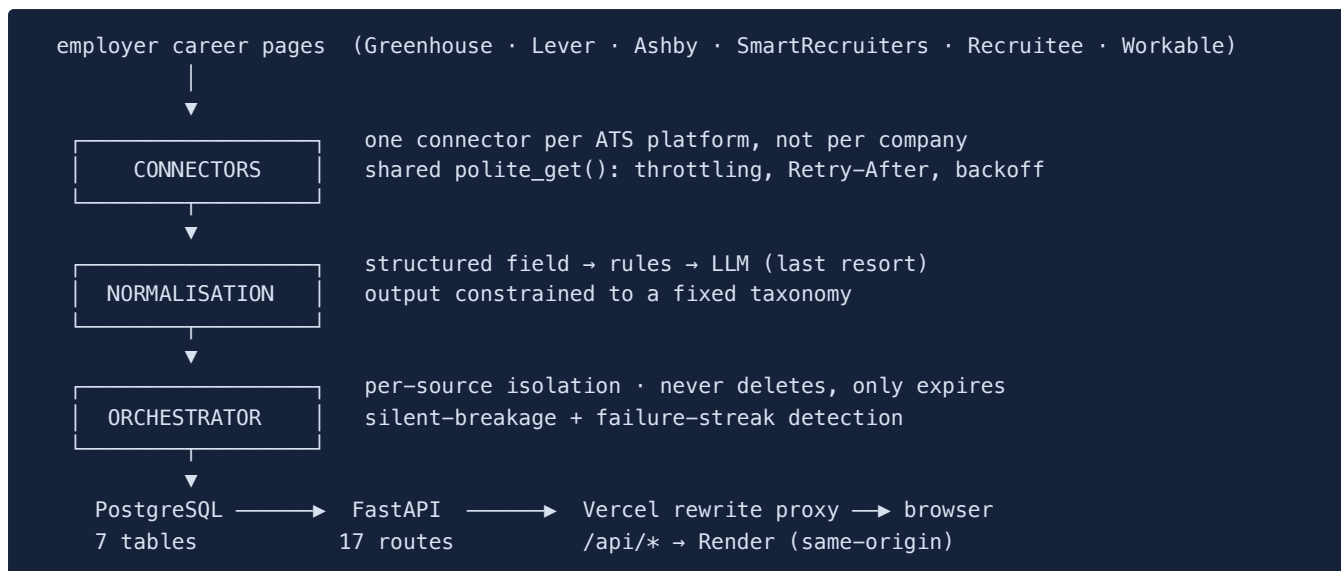
73
EMPLOYERS INDEXED

76
AUTOMATED CHECKS

2. Technology stack

Layer	Technology	Rationale
API	Python 3.12, FastAPI, Uvicorn	Async request handling; Pydantic schemas give typed responses and generated OpenAPI docs at no extra cost.
Data access	SQLAlchemy 2.0 ORM	One codebase runs on Postgres in production and SQLite locally, so the full pipeline is testable with no infrastructure.
Database	PostgreSQL 16 (Render)	Relational model with GIN full-text indexing for search.
HTTP client	httpx (async)	Concurrent connector fetches with per-domain rate limiting.
Auth	PyJWT, PBKDF2-HMAC-SHA256	200,000 iterations per password; stateless bearer tokens.
Frontend	Vanilla HTML/CSS/JS, single file	No build step and no framework runtime; the whole UI is one 1,016-line document that loads in a single request.
Hosting	Vercel (static + edge proxy), Render (API, Postgres, cron)	Frontend on a CDN; API and scheduled scrape co-located with the database.

3. Architecture



Connectors — one per platform, not per company

A single Greenhouse connector serves every company on Greenhouse by swapping the board token. Adding a company on an already-supported platform is a registry row, not code. All six connectors use *documented public job-board APIs* rather than scraping rendered HTML, which means no Terms of Service grey area, stable structured fields, immunity to career-page redesigns, and no anti-bot friction.

Politeness lives in the shared request path rather than in each connector: per-domain throttling, `Retry-After` compliance on HTTP 429, exponential backoff on 5xx, and a descriptive User-Agent identifying the crawler and a contact URL.

Normalisation — cheapest resolution first

Each field resolves through three tiers in order: the structured value the ATS already provided (free, authoritative), then rules and lookup tables (free, deterministic), then an LLM call (costs money, last resort). On the current registry the rules resolve **100% of departments with zero LLM calls**. The LLM path is opt-in via an API key; without it the pipeline runs rules-only, which is a valid production mode.

When the LLM is used, its output is hard-constrained to a fixed taxonomy of 15 departments and 10 seniority levels. A returned value outside those lists is discarded rather than stored — the model selects from an enumeration, it never invents a category.

Two deliberate calibration choices. Explicit level markers outrank inferred signals: "Engineer II" is Mid even if the description asks for eight years, but "Manager II" stays Manager — a role marker beats a level marker. Separately, seniority inferred from description text is capped at Senior, because Staff and Principal describe scope and influence rather than tenure; promoting a ten-year requirement to Staff would corrupt the level filter.

Orchestration — designed around the failure that actually happens

- **Per-source isolation.** One connector failing never blocks the run.
- **Jobs are never deleted.** Absent listings become `expired`. Deleting would destroy the time-to-fill and hiring-velocity data that later analytics depend on.
- **Reactivation preserves history.** A listing that returns keeps its original `first_seen_at`, so tenure stays honest.
- **Silent-breakage detection.** A career-page redesign rarely raises an error — it quietly returns zero jobs. A source dropping below half its recent typical volume raises an alert. This is the failure mode a plain try/except misses.

- **Failure streaks** alert after three consecutive failed runs.
- **Abandoned-run reaping.** A process killed mid-run leaves its status row marked `running` forever. Runs still open after 90 minutes are reconciled as failed, so health reporting and anomaly detection stay accurate.

4. Data model

Table	Purpose
companies	Registry: ATS type, board token, and the compliance flags <code>robots_allowed</code> / <code>tos_flag</code> checked before any connector is dispatched.
jobs	Normalised listings with full description, original <code>raw_payload</code> , content hash for change detection, and lifecycle timestamps.
scrape_runs	Per-company run history — status, counts, duration, errors. Feeds both health reporting and anomaly detection.
users	Accounts; PBKDF2 password hashes at 200,000 iterations.
saved_jobs , saved_searches	User collections.
outbound_clicks	Apply-click attribution, recorded on the outbound redirect.

`raw_payload` is retained on every job so normalisation can be reprocessed without re-scraping the source — the rules can be improved and replayed against historical data.

5. API surface

Endpoint	Function
GET <code>/api/jobs</code>	Filter by query, company, department, seniority, country, city, remote, recency; sort and paginate.
GET <code>/api/jobs/facets</code>	Live filter counts for the sidebar.
GET <code>/api/jobs/{id}</code>	Detail with full description.
GET <code>/api/jobs/{id}/similar</code>	Ranked by how many attributes match.
GET <code>/api/jobs/{id}/apply</code>	Records the click, then 302s to the employer's posting.
GET <code>/api/companies · /{slug}</code>	Companies with active-role counts.
GET <code>/api/stats</code>	Feed-wide counts and the true last-collection timestamp.
GET <code>/api/health</code>	Connector health and search-index state — not merely process liveness.
POST <code>/api/auth/signup · login</code>	Account creation and session issue.
<code>/api/me/saved-jobs · saved-searches · apply-profile</code>	Authenticated user collections.

6. Frontend

A scanning tool rather than a marketing page — dense rows, because the task is triaging many listings quickly. The interface is dark-first by deliberate choice: the product's central claim is that listings are current, and

luminance encodes recency more cheaply than hue. Every row carries a coloured edge tick that shifts green → amber → grey with age, so the freshness of the entire feed is readable at a glance without reading a single date.

- **Command palette** (⌘K) across companies, functions, levels, countries and loaded roles.
- **Faceted filtering** with live counts; all state syncs to the URL, so any filtered view is shareable and survives reload.
- **Saved roles** in local storage — no account required to bookmark.
- **List and grid views**, light/dark themes, skeleton loading states.
- **Detail panel** with full description, similar roles, and share links.
- **Accessibility**: keyboard navigation throughout, ARIA labelling, and `prefers-reduced-motion` support.

7. Engineering practices

Testing

76 automated checks — 19 end-to-end pipeline and 57 API — run offline against recorded fixtures, exercising the real connector parsers without network access. Tests cover insertion, idempotency, expiry, alerting, filtering, authentication, and the structural absence of any submit endpoint.

Regression tests are written to fail against the original defect, not merely to pass against the fix. The facet contract test, for example, feeds every facet value back through the filter it populates and asserts the returned count matches — a bug that returned zero results for every company filter is caught by that assertion and would not be caught by a shape check.

Performance

Search originally matched with `ILIKE` across job descriptions, which cannot use an index: **292 ms** on 6,400 rows, scanning 23 MB of text and degrading linearly. Replacing it with a PostgreSQL GIN full-text index reduced the same query to **5–9 ms end to end**. Company name is denormalised onto the jobs table specifically so it can sit inside the indexed vector — matching it against the joined table defeats the index, as a GIN lookup cannot combine with an OR against another relation.

Write batching was introduced after a production incident: a single employer's 794 new rows, carrying full descriptions and raw payloads, were committed as one oversized statement that the database dropped mid-write, taking every subsequent connection with it. Rows now flush every 100 inserts, and the connection pool verifies liveness on checkout.

Operational design

- Health endpoint reports connector status and search-index presence, so silent degradation is externally observable.
- Schema migrations and index creation run idempotently at startup; the registry self-seeds when empty.
- The scrape is fully idempotent — re-running after a partial failure completes rather than duplicating.
- Scrape concurrency is configurable, defaulting low, so it stays within what a small database instance tolerates.

8. Compliance posture

Every indexed source is a documented, public, unauthenticated job-board API that the platform publishes for exactly this purpose — surfacing a company's open roles. The registry records `robots_allowed` and `tos_flag`

per company, and the orchestrator refuses to dispatch a connector for any source flagged either way. That gate is enforced in code, not advisory.

Aggregators and reposting job boards are deliberately excluded from the registry even where their ATS token resolves — one such source alone would have contributed several thousand listings, but indexing a reposter would contradict the product's central claim that every listing comes from the employer.

9. Current status and roadmap

Area	State
Deployment	Live. Frontend on Vercel CDN, API and Postgres on Render, daily scrape on a scheduled cron.
Coverage	73 employers verified against live APIs; a further 46 identified and ready to add.
Search	Full-text indexed; sub-10 ms at current volume.
Infrastructure	Free tiers. Requires paid plans before production traffic — the free database expires and the free web service sleeps after inactivity.

Planned next

- **Server-side rendering.** Organic search for "[Company] [Role] jobs" is the primary acquisition channel for this category, and the current client-rendered pages are invisible to crawlers. A Next.js rewrite with `JobPosting` structured data is the highest-value work outstanding.
- **HTTP caching and rate limiting** — job data changes once daily, so CDN caching would absorb most traffic; the API is currently unmetered.
- **Registry expansion.** Roughly 28% of candidate ATS tokens resolve; broader coverage requires supporting Workday and similar platforms that publish no open job-board API.
- **Payload archival** — raw payloads dominate storage and can move to object storage beyond a retention window.